

创新点：三级硬件复用把级联开销压到 KPI 之内

背景：外码 RS 抗突发、内码 BCH 纠随机错的级联码能带来编码增益，但 内码译码是级联链路上 **多出来的时延**——若不优化，会超过导师给定的 10% 时延 KPI 红线。



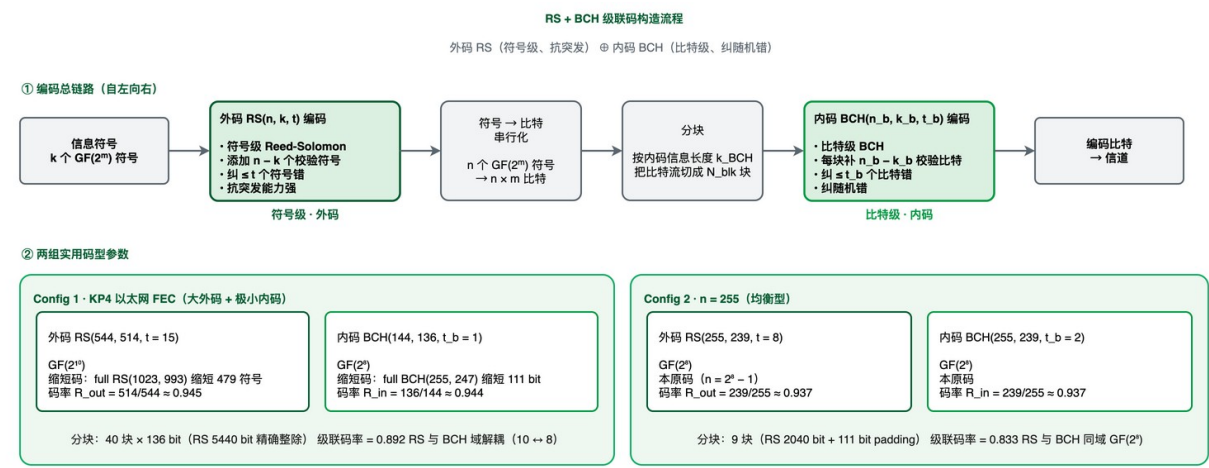
三个创新点逐项叠加后的 KPI 结果

Config	Pure RS-BM	+Cascade(Conv)	+Direct	+Lagrange v1	+Lagrange v2
Config 1 (KP4)	33 cyc （基线）	37 cyc +12.1% ✗	35 cyc +6.1% ✓	34 cyc +3.0% ✓	33 cyc +0.0% ✓
Config 2 (n=255)	19 cyc （基线）	27 cyc +42.1% ✗	22 cyc +15.8% ✗	21 cyc +10.5% ✗	20 cyc +5.3% ✓

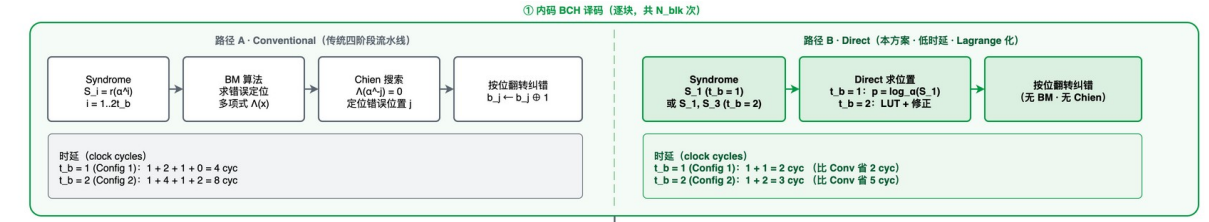
两个 config 的达标路径不同：Config 1 外码大 (t=15)、内码极小 (t=1)，仅 **Direct** 一项 即可达标；Config 2 时延基数小 (19cyc)、内码开销占比更重，必须 三项创新齐全 才达标（KPI ≤10%）。外码越大、内码越小，级联时延开销越可忽略。

级联码构造原理与译码原理

① 码构造：外码 **RS**（符号级）⊕ 内码 **BCH**（比特级），两组实用码型



② 内码 **BCH** 译码：Conventional（传统）vs Direct（查表求根，本方案）



③ 抗突发：三步矩阵交织（复现 **APCC 2022**，接入 **Config 1**）



Direct 译码原理：闭式代数变换 + 查表求根

思路：传统 BM 译码需迭代 $2t$ 次求错误定位多项式 $\Lambda(x)$ ，再做 Chien 搜索遍历所有位置。Legendijk 论文指出： $t \leq 4$ 时， $\Lambda(x)$ 的根可以用 **闭式代数变换 + LUT 查表** 直接求出，省掉迭代和遍历。

t=1（退化闭式）： $S_1=0 \rightarrow$ 无错；否则错误位置 $p = \log_{\alpha}(S_1)$ ，直接查 α 幂表后修正 1 个比特。共 2 cycles：syndrome + (log 查表 + 修正)。

t=2（通用情况）：代数变换分 3 个 cycle 完成求根



关键性质 (char-2 域)：在 $GF(2^k)$ 上，若 Y_1 是 $A(Y)$ 的一个根，则 $Y_2 = Y_1 + 1$ 也一定是根——这是二次多项式在特征 2 域上的固有性质。因此只需一个 2^k 大小的 **LUT** (而非两个独立 LUT) 即可同时得到两个根，LUT 可完全展开为组合逻辑，1 个 clock cycle 内完成。

对比：Conventional 需 BM 迭代 $2t$ 次求 $\Lambda(x)$ + Chien 搜索遍历 n 个位置 $\rightarrow t=2$ 需 8 cycle；Direct 用代数变换替代迭代、单次 LUT 替代遍历 $\rightarrow t=2$ 仅 3 cycle， $t=1$ 仅 2 cycle。

拉格朗日共享原理：硬件模块复用逐级压缩级联时延

说明：这里的“拉格朗日共享”指 **硬件模块复用**（内外码共享同一套 syndrome 计算单元 / GF 乘法器阵列），不是拉格朗日插值运算本身——BCH-Direct 用查表求根，RS 仍用 BM 迭代，共享的是电路，不是算法。



延伸：LUT 规模也影响可压缩空间

GF 域	LUT 大小	查表 MUX 深度	Direct 总周期
n=255 (GF(2 ⁸))	256 项	8-input	3 cycle
n=127 (GF(2 ⁷))	128 项 (减半)	7-input (更浅)	2 cycle (precompute+LUT+修正合并 1cycle)

时延计算依据：公式代入两组 config 得到逐项 cycle 数

依据：下表把 前两页原理公式（RS-BM(t)=2t+3、BCH-Direct(t)、v0/v1/v2 共享）逐项代入 Config1(KP4) 与 Config2(n=255) 的实际参数，得到与第 1 页 KPI 表、第 6 页柱状图完全对应的 cycle 数——不是仿真测出来的，是公式算出来的。

Config 1（KP4）：RS(544,514,t=15) + BCH(144,136,t=1) | Config 2（n=255）：RS(255,239,t=8) + BCH(255,239,t=2)

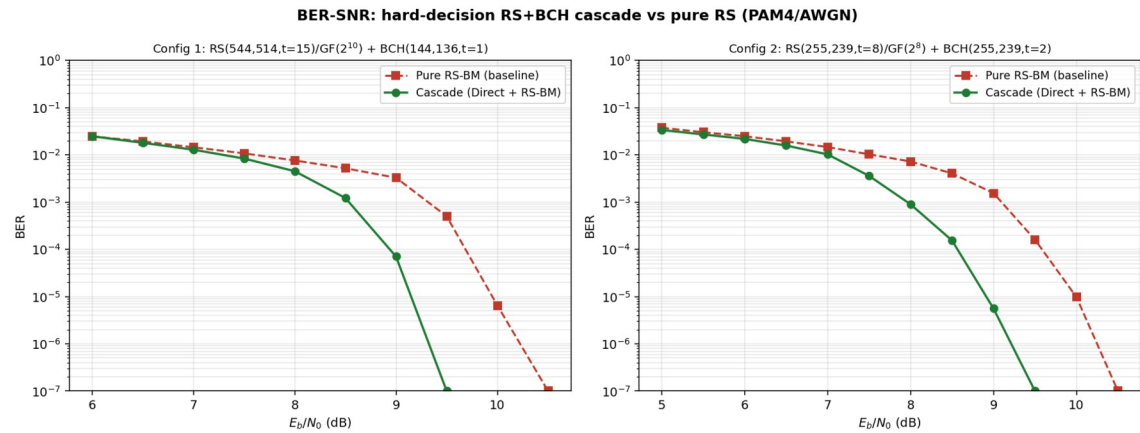
方案	公式	Config1 代入 (t_rs=15, t_bch=1)	cyc	Config2 代入 (t_rs=8, t_bch=2)	cyc
Pure RS-BM（基线）	$T = 2t_{rs}+3$	$2 \times 15 + 3$	33	$2 \times 8 + 3$	19
+Cascade（BCH-Conv, v0）	$T = T_{BCH-Conv} + T_{RS-BM}$	$4 + 33$	37	$8 + 19$	27
+Direct（BCH-Direct, v0）	$T = T_{BCH-Direct} + T_{RS-BM}$	$2 + 33$	35	$3 + 19$	22
+Lagrange v1（共享 syndrome）	$T = T_{BCH-Direct} + T_{RS-BM} - 1$	$35 - 1$	34	$22 - 1$	21
+Lagrange v2（再共享乘法器阵列）	$T = T_{BCH-Direct} + T_{RS-BM} - 2$	$35 - 2$	33	$22 - 2$	20

两组 config 读法不同：Config1 基线 33cyc 大、内码 t_bch=1 开销小，Direct 一项代入后 35→34→33，第三步已回到基线（+0.0%），KPI 早早达标；Config2 基线 19cyc 小、内码 t_bch=2 开销占比更重，同样代入 v0/v1/v2 公式，35cyc 起点的 Conv/Direct 差距在小基线下被放大，必须代入到 v2（-2）才压到 20cyc（+5.3%，达标）。

* T_RS-BM、T_BCH-Conv/Direct 公式见「构造原理」「Direct 译码原理」「拉格朗日共享原理」三页；本页只做数值代入，不重复推导。

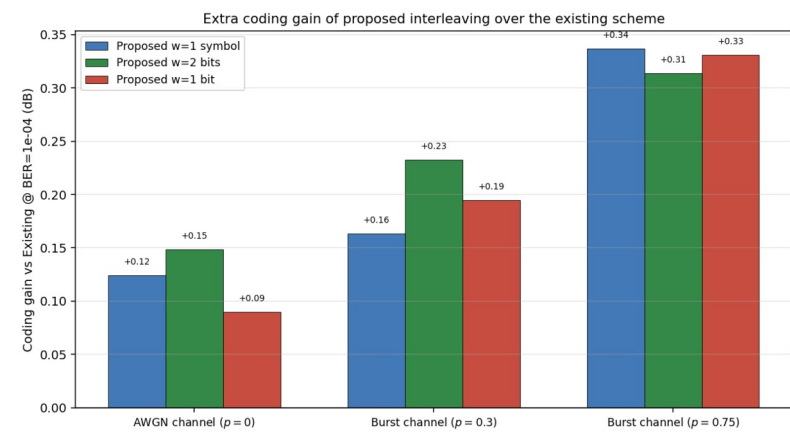
结果：编码增益、突发抗性

① BER-SNR：级联 vs 纯 RS（两组码型）



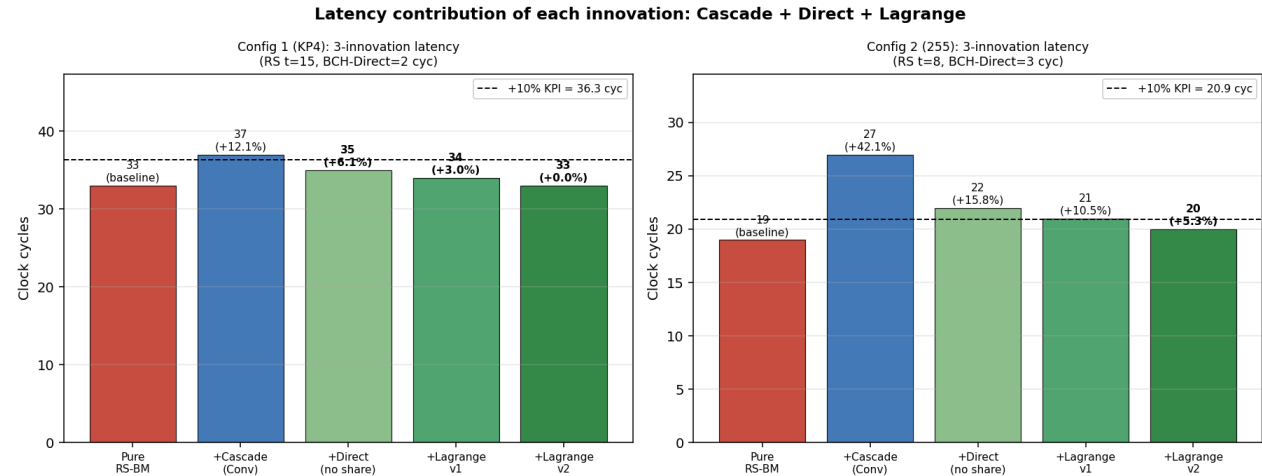
BER=10⁻⁴：Config1 级联≈ 8.9dB vs 纯 RS≈9.7dB(+0.7dB)；Config2 级联≈ 8.6dB vs 纯 RS≈9.6dB(+1.0dB)

② 矩阵交织抗突发增益（vs existing 符号交织）



AWGN 增益小 (+0.09~0.15dB)；突发 p=0.75 增益最大 (+0.31~0.34dB)——交织在突发信道下价值最明显

③ 三创新点逐项时延贡献（Cascade → Direct → Lagrange v1/v2）



时延分析、复杂度分析与硬件资源开销

① 时延模型（clock cycles）

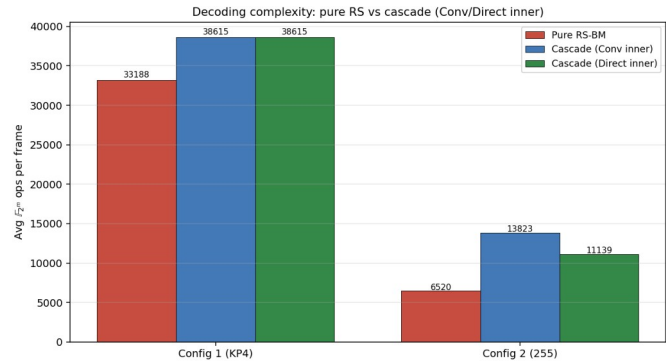
译码器	时延分解	cyc
RS-BM(t)	syndrome+2t·BM+Chien+Forney	2t+3
BCH-Conv(t=1)	syndrome+ELP+Chien+correct	4
BCH-Conv(t=2)	1+2t+1+margin	8
BCH-Direct(t=1)	syndrome+(log 查表 + 修正)	2
BCH-Direct(t=2)	syndrome+precompute+LUT	3

三档共享：T_none→T_v1(-1, 共享 syndrome)→T_v2(-2, 再共享 GF 乘法器阵列)

③ 硬件资源开销：GF 乘法器数量 · LUT 大小

硬件资源	Config 1 (KP4, GF(2 ¹⁰)/GF(2 ⁸))	Config 2 (n=255, GF(2 ⁸))	说明
GF 乘法器阵列	n 路并行（RS: 544 路 / BCH: 144 路，两级不共享物理阵列——不同域）	n=255 路并行（RS/BCH 同域 GF(2 ⁸ ），v2 可分时复用同一套）	Chien 搜索 + Direct 求值均需并行求值 $\Lambda(\alpha^*)$
BCH-Direct LUT 大小	2 ⁸ = 256 项（t=1 退化为 log 表，非二次查表）	2 ⁸ = 256 项 × (2 根 / 项) = 512 条目	t=2：k=X ² +X 索引，存 (X,X+1)
LUT 存储 (ROM/BRAM)	log 表 256×8bit ≈ 2 kbit（复用 α 幂表）	256×(2×8bit) ≈ 4 kbit	n 越小 LUT 越小，MUX 深度越浅
共享后节省	Direct 比 Conv 硬件面积降约 62%（Lagendijk 2026）	同左 + syndrome/ 乘法器阵列分时复用省 2 cyc	面积节省来自去掉 BM 迭代器 + Chien 专用阵列

② 复杂度：每帧平均 $\mathbb{F}_2$ m 运算量



Direct 相对 Conv 省的是周期数（时延），总运算量 (ops) 接近——t 小时 Chien 只贡献 O(t) 个乘法。Direct 价值在关键路径深度，不在总运算量。

结论：Direct + Lagrange 硬件共享是唯一能同时满足两组 config 10% 时延 KPI 的方案；总运算量 (ops) 几乎不受 Direct 影响，硬件收益集中在关键路径深度与面积（LUT/ 乘法器阵列复用），而非算力总量。